

4.1. Python

Tema 4. Implementación y aplicación

Javier Paris

Table of contents

1	Introducción a Python	2
1.1	¿Qué es Python?	2
1.2	Motivación	3
1.3	Usos	4
2	Entorno	4
2.1	Instalación	4
2.2	Intérprete	5
2.3	Fichero python	5
2.4	Jupyter Notebook	5
2.5	Google Colab	6
3	Bases del lenguaje	7
3.1	Variables	7
3.2	Asignación	7
3.3	Comentarios	8
3.4	Tipos de datos	8
3.5	Casting	8
3.6	Operadores aritméticos	9
3.7	Operadores lógicos	9
3.8	Operadores de comparación	10
4	Tipos de datos	10
4.1	Listas	10
4.2	Listas - Acceso a elementos	10
4.3	Listas - Modificación	11
4.4	Tuplas	11
4.5	Sets	12
4.6	Diccionarios	12

4.7	Strings	12
4.8	String - Métodos	13
5	Control de flujo	13
5.1	Bloques	13
5.2	Condicionales	14
5.3	Bucle while	14
5.4	Bucle for	14
5.5	Bucle for - range	15
5.6	Operadores de bucle	15
6	Funciones	15
6.1	Librerías	15
6.2	Funciones	16
6.3	Funciones - Argumentos	16
6.4	Funciones - Retorno	16
6.5	Built-in functions	17
7	Clases	17
7.1	Clases	17
7.2	Clases - Métodos	17
7.3	Clases - built-in methods	18
8	Avanzado	18
8.1	List comprehension	18
8.2	Lambda	18
8.3	Enumerate	19
8.4	Trabajar con diccionarios	19
8.5	Iterar sobre más de un valor	20
8.6	Herencia	20
8.7	Mucho más	21

1 Introducción a Python

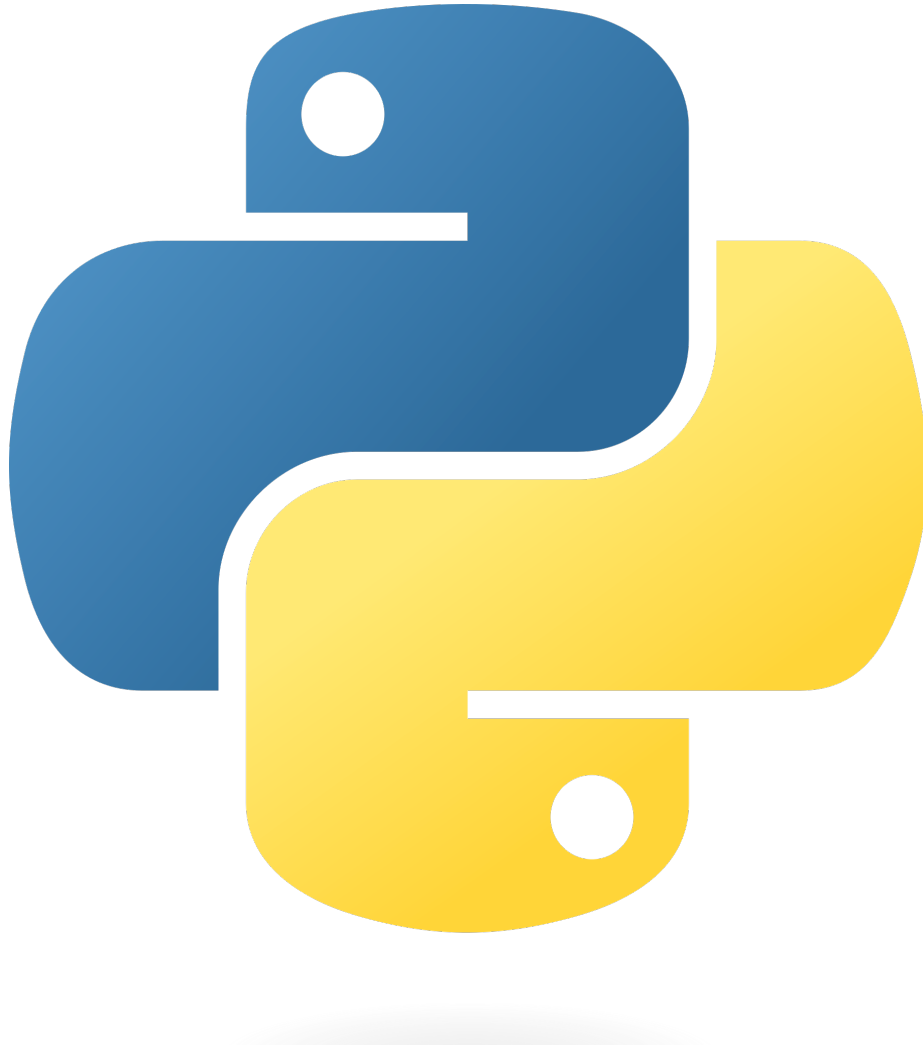
1.1 ¿Qué es Python?

Python es un lenguaje **interpretado** de **alto nivel**. Es un lenguaje **multiparadigma** ya que soporta **orientación a objetos**, **programación imperativa** y, en menor medida, **programación funcional**. Es un lenguaje **dinámicamente tipado** y **multiplataforma**.

- **No compilado:** Interpretado
- **Alto nivel:** Fácil y abstracto

- **Dinámicamente tipado:** Variables no están sujetas a un tipo (*nombres*)
- **IA:** numpy, pandas, scikit-learn, tensorflow, etc.

<https://python.org/>



1.2 Motivación

Python es el lenguaje de programación más popular: <https://spectrum.ieee.org/top-programming-languages-2024>.

1.2.1 Pros

- Fácil de aprender
- Código compacto y entendible
- Ampliamente utilizado (cuenta con amplio soporte)
- Muchas librerías

1.2.2 Cons

- Muy alto nivel
- Lento (comparado con C)
- No es fuertemente tipado

1.3 Usos

Python es usado en muchos campos, como:

- **Análisis de datos:** Pandas, Numpy, etc.
- **Inteligencia Artificial:** Machine Learning, Deep Learning, etc.
- **Desarrollo web:** Django, Flask, etc.
- **Automatización:** Scripts, bots, etc.
- **Industria:** Automatización, IoT, etc.
- **Juegos:** Pygame, Panda3D, etc.
- etc.

2 Entorno

2.1 Instalación

Existen diversas formas de trabajar con Python, que dependen del entorno que se quiera utilizar, del uso que se le vaya a dar, y del sistema operativo.

- **Local:** Instalación en tu propio ordenador
 - **Anaconda:** Distribución de Python que incluye librerías y entorno de desarrollo.
 - **pip:** Instalador de paquetes de Python.
- **Online:** Plataformas online como Google Colab, Jupyter Notebook, etc.

2.2 Intérprete

Python es un lenguaje interpretado, lo que significa que el código se ejecuta línea a línea. Se puede utilizar Python como un **intérprete interactivo** donde se ejecuta una línea de código a la vez.

```
print("Hello World!")
```

Hello World!

Esto se suele hacer desde una terminal de comandos:

```
python
```

2.3 Fichero python

El uso más común de Python es escribir un *script* en un fichero con extensión `.py`. Estos ficheros se ejecutan a través del intérprete de Python, que lee el código línea a línea y lo ejecuta.

```
# Fichero: hello.py  
s = "Hello World!"  
print(s)
```

Hello World!

Para ejecutar el fichero, se hace desde la terminal:

```
python hello.py
```

2.4 Jupyter Notebook

Jupyter Notebook es un formato de documento que permite combinar texto y gráficos con código Python. Esta es la herramienta más extendida en el ámbito de la ciencia de datos y el machine learning.

- **Interactivo:** Se ejecuta celda a celda
- **Visual:** Muestra gráficos y texto
- **Exportable:** A HTML, PDF, etc.



2.5 Google Colab

Google Colab es una plataforma gratuita de Google que permite ejecutar cuadernos de Jupyter en la nube. Esto tiene muchas ventajas;

- **Notebook:** trabaja por defecto con cuadernos de Jupyter
- **Online:** no requiere de instalación ni actualización
- **Nube:** es independiente del dispositivo (no dará problemas de hardware ni compatibilidad)
- **Dependencias:** incluye muchas librerías preinstaladas con las mismas versiones para todos los usuarios

! Important

Esta es la plataforma recomendada para el curso.



3 Bases del lenguaje

3.1 Variables

Las variables en Python son **dinámicamente tipadas**, lo que significa que no se especifica el tipo de variable al declararla.

```
x = 5      # int
y = "Hello" # str
print(x, y)
```

5 Hello

3.2 Asignación

Se asigna un valor a una variable con el operador =.

```
x = 5 # x is 5
y = x # y is 5
x = 3 # x is 3
a, b = x, 2 # a is 3, b is 2
print(x, y, a, b)
```

3 5 3 2

3.3 Comentarios

Todo lo escrito detrás de un `#` o entre `"""` es un comentario y no se ejecuta.

```
x = 5
# This is a comment
# x = 3
""" x = 2 """
print(x)
```

5

3.4 Tipos de datos

Python tiene varios tipos de datos básicos:

- Numéricos: `int`, `float`
- Texto: `str`
- Booleanos: `bool`
- Secuencias: `list`, `tuple`
- Conjuntos: `set`
- Diccionarios: `dict`

```
x = 5          # int
y = 5.0        # float
z = "Hello"    # str
print(type(x), type(y), type(z))
```

<class 'int'> <class 'float'> <class 'str'>

3.5 Casting

Se puede cambiar el tipo de una variable con *casting*.

```
x = 5          # int
y = str(x)     # str
print(y, type(y))
```

5 <class 'str'>

3.6 Operadores aritméticos

```
x = 5    # int
y = 2    # int
print(x + y) # Add
print(x - y) # Subtract
print(x * y) # Multiply
print(x / y) # Divide
print(x % y) # Modulus
print(x ** y) # Exponentiation
print(x // y) # Floor division
```

```
7
3
10
2.5
1
25
2
```

3.7 Operadores lógicos

```
x = True    # bool
y = False   # bool
print(not x) # Not
print(x and y) # And
print(x or y) # Or
```

```
False
False
True
```

0, string vacío, lista vacía, etc. son considerados **False** en Python.

```
x = 0    # int -> False
y = ""   # str -> False
print(bool(x), bool(y))
```

```
False False
```

3.8 Operadores de comparación

```
x = 5 # int
y = 2 # int
print(x == y) # Equal
print(x != y) # Not equal
print(x > y) # Greater than
print(x >= y) # Greater than or equal
print(x < y) # Less than
print(x is y) # Same object
```

```
False
True
True
True
False
False
```

4 Tipos de datos

4.1 Listas

Las listas son una colección ordenada y mutable de elementos.

```
l = [1, 2, 3, 4, 5] # list
print(l)
```

```
[1, 2, 3, 4, 5]
```

4.2 Listas - Acceso a elementos

Para acceder a un elemento de una lista, se utiliza el operador `[]`.

```
l = [1, 2, 3, 4, 5]
print(l[0]) # First element
print(l[-1]) # Last element
print(l[1:3]) # Slice [2,3]
```

```
1
5
[2, 3]
```

4.3 Listas - Modificación

Los elementos en las listas se pueden modificar accediendo mediante []. Las listas también pueden ampliarse `append` y sumarse `+`.

```
l = [1, 2] # list
print(l)
l[0] = 0   # l = [0,2]
print(l)
l.append(3) # l = [1,2,3]
print(l)
l = l + l   # l = [1,2,3,1,2,3]
print(l)
```

```
[1, 2]
[0, 2]
[0, 2, 3]
[0, 2, 3, 0, 2, 3]
```

4.4 Tuplas

Las tuplas son una colección ordenada e inmutable de elementos. Se accede a ellas como a las listas, pero no se pueden modificar.

```
t = (1, 2, 3, 4, 5)
print(t)
print(t[1:3])
```

```
(1, 2, 3, 4, 5)
(2, 3)
```

4.5 Sets

Los sets (o conjuntos) son una colección desordenada y mutable de elementos únicos. No se puede acceder a los elementos salvo por iteración, pero se pueden añadir y eliminar elementos.

```
s = {1, 2, 3, 4, 5}
print(s)
s.add(6)
print(s)
s.add(6)
print(s)
s.remove(1)
print(s)
```

```
{1, 2, 3, 4, 5}
{1, 2, 3, 4, 5, 6}
{1, 2, 3, 4, 5, 6}
{2, 3, 4, 5, 6}
```

4.6 Diccionarios

Los diccionarios son una colección no ordenada de pares clave-valor.

```
d = {"a": 1, "b": 2, "c": 3}
print(d)
print(d["a"])
d["a"] = 0
print(d)
```

```
{'a': 1, 'b': 2, 'c': 3}
1
{'a': 0, 'b': 2, 'c': 3}
```

4.7 Strings

Los strings son tuplas de caracteres con métodos específicos.

```
s = "Hello World!"
print(s)
print(s[0])
print(s[1:5])
```

```
Hello World!
H
ello
```

4.8 String - Métodos

```
s = "Hello World!"
print(s.upper())
print(s.lower())
print(s.replace("H", "J"))
print(s.split(" "))
print(s.find("W"))
```

```
HELLO WORLD!
hello world!
Jello World!
['Hello', 'World!']
6
```

5 Control de flujo

5.1 Bloques

Los bloques de código en Python se definen por la indentación (espacios y tabuladores al principio de la línea). Por eso, es muy importante mantener la indentación correcta.

```
if True:
    print("Bloque 1")
    if False:
        print("Bloque 2")
print("Bloque 3")
```

```
Bloque 1
Bloque 3
```

5.2 Condicionales

Los condicionales en Python se realizan con `if`, `elif` y `else`.

```
x = 5
if x > 5:
    print("Mayor que 5")
elif x < 5:
    print("Menor que 5")
else:
    print("Igual a 5")
```

Igual a 5

5.3 Bucle while

El bucle `while` se ejecuta mientras la condición sea verdadera.

```
x = 0
while x < 5:
    print(x)
    x += 1
```

0
1
2
3
4

5.4 Bucle for

El bucle `for` se utiliza para iterar sobre una secuencia.

```
l = [1, 2, 3, 4, 5]
for x in l:
    print(x)
```

1
2
3
4
5

5.5 Bucle for - range

La función `range` genera una secuencia de números.

```
for x in range(5):  
    print(x)
```

```
0  
1  
2  
3  
4
```

5.6 Operadores de bucle

El operador `break` se utiliza para salir de un bucle. El operador `continue` se utiliza para saltar a la siguiente iteración.

```
for x in range(5):  
    if x % 2:  
        continue  
    if x == 4:  
        break  
    print(x)
```

```
0  
2
```

6 Funciones

6.1 Librerías

Python tiene una gran cantidad de librerías que se pueden importar y utilizar. Para importar se usa la palabra clave `import`. Se puede renombrar la librería con `as`.

```
import numpy          # To use numpy use numpy.function()  
import numpy as np    # To use numpy use np.function()
```

6.2 Funciones

Las funciones en Python se definen con la palabra clave `def`. Estas sirven para encapsular código y reutilizarlo.

```
def my_function():  
    print("Hello from a function")  
  
my_function()  
my_function()
```

```
Hello from a function  
Hello from a function
```

6.3 Funciones - Argumentos

Las funciones pueden tener argumentos que se pasan al llamar a la función. Los argumentos pueden usarse por posición o por nombre. También se pueden definir argumentos por defecto, que se usan si al llamar a la función no se especifican.

```
def my_function(name, age=25):  
    print(f"Hello {name}, you are {age} years old")  
  
my_function("Alice")  
my_function(age=30, name="Bob")
```

```
Hello Alice, you are 25 years old  
Hello Bob, you are 30 years old
```

6.4 Funciones - Retorno

Las funciones pueden devolver un valor con la palabra clave `return`.

```
def my_function(x):  
    return x * 2  
  
print(my_function(3))  
print(my_function(5))
```

```
6  
10
```

6.5 Built-in functions

Estas son algunas de las funciones integradas en Python que se utilizan con frecuencia:

- `print`: Imprime por pantalla
- `range`: Devuelve un iterable con una secuencia de números
- `len`: Devuelve la longitud de un iterable o contenedor
- `input`: Lee por teclado
- `sum`: Devuelve la suma de los elementos de un iterable
- `min`: Devuelve el mínimo de los elementos de un iterable
- `max`: Devuelve el máximo de los elementos de un iterable
- `any`: Devuelve `True` si algún elemento de un iterable es `True`
- `all`: Devuelve `True` si todos los elementos de un iterable son `True`
- `sorted`: Devuelve una lista ordenada de los elementos de un iterable

7 Clases

7.1 Clases

Para crear estructuras de datos más complejas, se pueden utilizar clases. La palabra clave `class` se utiliza para definir una clase.

```
class MyClass:
    x = 5

c = MyClass()
print(c.x)
```

5

7.2 Clases - Métodos

Las clases pueden tener métodos, que son funciones que pertenecen a la clase. El primer argumento de un método es el objeto que lo llama, por convención se llama `self`.

```
class MyClass:
    def my_method(self):
        print("Hello from a method")

c = MyClass()
c.my_method()
```

Hello from a method

7.3 Clases - built-in methods

Existen varios métodos especiales que se pueden definir en una clase:

- `__init__`: Constructor de la clase
- `__str__`: Representación en string de la clase
- `__len__`: Longitud de la clase
- `__getitem__`: Acceso a elementos a través de `[]`
- etc.

8 Avanzado

8.1 List comprehension

List comprehension es una forma rápida y compacta de crear listas.

```
l1 = [x for x in range(5)]
print(l1)

l2 = [x for x in range(5) if x % 2]
print(l2)
```

[0, 1, 2, 3, 4]
[1, 3]

8.2 Lambda

Lambda es una forma de definir funciones anónimas en una sola línea.

```
f = lambda x: x * 2
print(f(3))

# Sort by second element
l = [(1, 2), (3, 1), (5, 3)]
l.sort(key=lambda x: x[1])
```

6

8.3 Enumerate

`enumerate` es una función que devuelve un iterable con índices y valores.

```
l = ["a", "b", "c"]
for i, x in enumerate(l):
    print(i, x)
```

```
0 a
1 b
2 c
```

8.4 Trabajar con diccionarios

Los diccionarios se pueden recorrer por clave, valor o ambos.

```
d = {"a": 1, "b": 2, "c": 3}
for k in d.keys():
    print(k)

for v in d.values():
    print(v)

for k, v in d.items():
    print(k, v)
```

```
a
b
c
1
```

```
2
3
a 1
b 2
c 3
```

8.5 Iterar sobre más de un valor

Se pueden crear bucles que iteren sobre más de un iterable a la vez, cuando el iterable es una colección. La función `zip` crea un iterable que agrupa los elementos de los iterables que se le pasan.

```
l1 = [1, 2, 3]
l2 = ["a", "b", "c"]
for x, y in zip(l1, l2):
    print(x, y)
```

```
1 a
2 b
3 c
```

8.6 Herencia

La herencia es una característica de la programación orientada a objetos que permite crear una nueva clase a partir de una clase existente.

```
class Parent:
    def __init__(self, x):
        self.x = x

class Child(Parent):
    def __init__(self, x, y):
        super().__init__(x)
        self.y = y

c = Child(1, 2)
print(c.x, c.y)
```

```
1 2
```

8.7 Mucho más

El lenguaje Python es muy extenso y tiene muchas más características y funcionalidades. En este tema se ha hecho un resumen básico de las ideas principales del lenguaje y los elementos más utilizados.

Animamos a los estudiantes a explorar más sobre el lenguaje y a profundizar en las áreas que más les interesen.

